

DAT510 Assignment 1

Dirhad Hamid Moussa

September 2026

Contents

1	Abstract	2
2	Cryptanalysis of primitive ciphers	3
2.1	Part I	3
2.1.1	Deciper	3
2.1.2	Strategy	4
2.2	Part II	5
2.2.1	Simplified DES (SDES)	5
2.2.2	Triple SDES	6
2.2.3	Cracking SDES (ctx1)	6
2.2.4	Cracking Triple SDES (ctx2)	6
3	Implement Secure Communications	8
3.0.1	Diffie-Hellman Key Exchange	8
3.0.2	Deriving a Symmetric Key with BBS	8
3.0.3	Symmetric Encryption with SDES	9
3.0.4	Results	9
4	Conclusion	10

Chapter 1

Abstract

In this assignment we explore classical cryptographic techniques through three tasks. In part I, a polyalphabetic substitution cipher was broken using frequency analysis to recover the key length and the individual substitution alphabets. In part II, the Simplified SDES block cipher and its Triple SDES were implemented from scratch, and then used to brute-force two ciphertexts recovering 10-bit key for the SDES message and two 10-bit keys for the Triple SDES message. Finally, a secure communication system between Alice and Bob was built by combining three primitives: a Diffie-Hellman key exchange to establish a shared secret, a Blum Blum Shub CSPRNG to derive a symmetric key from the secret, and SDES for encryption.

Chapter 2

Cryptanalysis of primitive ciphers

2.1 Part I

2.1.1 Deciper

The main goal for this problem was to decipher a text using polyalphabetic substitution, where the key length was no larger than 6 and where blank spaces first were deleted and then inserted at random locations. We also got to know that it uses an English alphabet consisting of 26 letters A through Z and some letter mappings for example A to X and B to Y.

The blank text we were given was:

```
FRRUU OIIYE AMIRN QLQVR BOKGK NSNQQ IUTTY
IIYEA WIJTG LVILA ZWZKT ZCJQH IFNYI WQZXH
RWZQW OHUTI KWNNQ YDLKA EOTUV XELMT SOSIX
JSKPR BUXTI TBUXV BLNSX FJKNC HBLUK PDGUI
IYEAM OJCXW FMJVM MAXYT XFLOL RRLAA JZAXT
YYWFY NBIVH VYQIO SLPXH ZGYLH WGFSX LPSND
UKVTR XPKSS VKOWM QKVCR TUUPR WQMWY XTYLQ
XXXTR TJJGO OLMXV CPPSL KBSEI PMEGC RWZRI
YDBGE BTMFP ZXVMF MGPVO OKZXX IGGFE SIBRX
SEWTY OOOKS PKYFC ZIEYF DAXKG ARBIW KFWUA
SLGLF NMIVH VVPTY IJNSX FJKNC HBLUK PDGUI
IYEAM HVFDY CULJS EHHMX LRXBN OLVMR
```

I managed to decipher this text to become:

```
CRYPTOGRAPHY CAN BE STRONG OR WEAK. CRYPTOGRAPHIC STRENGTH IS MEASURED IN THE TIME A
ND RESOURCES IT WOULD REQUIRE TO RECOVER THE PLAIN TEXT. THE RESULT OF STRONG CRYPTOGR
APHY IS CIPHER TEXT THAT IS VERY DIFFICULT TO DECIPHER WITHOUT POSSESSION OF THE APP
ROPRIATE DECODING TOOL. HOW DIFFICULT GIVEN ALL OF TODAY'S COMPUTING POWER AND A VA
```

LABLETIMEEVENABILLIONCOMPUTERSDOINGABILLIONCHECKSASECONDITISNOTPOSSIBL
ETODECIPHERTHERESULTOFSTRONGCRYPTOGRAPHYBEFORETHEENDOFTHEUNIVERSE

The keyword used was DATFBA, which is fully readable English.

2.1.2 Strategy

I started off by cleaning up the ciphered text, removing line breaks and empty spaces. I then counted the frequency of each letter and found that the distribution was fairly even.

Next, I checked the probability of two identical letters appearing after each other by calculating the index of coincidence for each letter:

$$IC = \frac{\sum Counter(A) \cdot (Counter(A) - 1)}{len(cipher) \cdot (len(cipher) - 1)}$$

where **Counter(A)** is the count of letter A and **len(cipher)** is the total length of the ciphertext. I calculated the average IC for key lengths ranging from 1 to 6 by splitting the ciphertext into columns based on the key length L, and averaging the IC across the resulting column set. The goal was to estimate the correct key length, since a column corresponding to a true key should behave like a monoalphabetic ciphertext and produce an IC close to that of English text (≈ 0.067).

IC-analysis :

L = 1 0.0393

L = 2 0.0394

L = 3 0.0398

L = 4 0.0403

L = 5 0.0393

L = 6 0.0404

As we can see from the IC-analysis, the values range from 0.039 to 0.040, which is noticeably lower than the IC of English text (≈ 0.067) for every tested key length. This made it difficult to determine the correct key length from IC alone, so I also performed a Kasiski examination to try to confirm it. By analysing the ciphertext, I found that the sequence "FJKNCHBLUKPDGUIIYEAM" repeats at two positions, 245 characters apart. The distance 245 is only evenly divisible by 1 and 5 within the range of possible key lengths (1-6), suggesting a key length of 5. However, this contradicted the IC-analysis, where L=6 produced the highest average IC (0.0404) among the tested lengths, making it the strongest candidate despite the Kasiski result.

This inconsistency suggested that the cipher might not be a reaping-key Vigenere cipher, where Kasiski examination is expected to reveal the key length through repeated patterns. Instead, it pointed towards an autokey cipher, where only the first m characters form the actual key and the remainder of the key is derived from the plaintext itself.

Therefor i implemented and autokey decryption approach for each dandidate key length, using a chi-squared test against the expected English letter frequency to score each result. A lower chi-squared score indicates a closer match to natural english text. Testing with $m = 6$ produced fully readable English plaintext, confirming both the key length and the auto key structure of the cipher.

To implement the autokey decryption, i first split the ciphertext into m "chains" based on position modulo m , for each key position r from 0 to $m-1$, i extracted every m -th character starting at r . In an autokey cipher, only the very first character of each chain is encrypted using the true key letter, every subsequent character in the chain is instead encrypted using the previously decrypted plaintext letter as the key. This means that once the first letter of a chain is correctly guessed, the entire chain can be decrypted sequentially, since each step only depends on the letter before it.

For each chain i tried all 26 possible values for the first key letter, decrypted the full chain for each guess, and scored the resulting text using chi-squared to test against the expected english letter frequncies. This process was repeated for all m chains, and the result were interleaved back into their original position to reconstruct the full plaintext.

Running this approach with $m=6$ produced fully readable English plaintext, confirming that six was indeed the correct key length, and the keyword DATFBA.

2.2 Part II

2.2.1 Simplified DES (SDES)

I implemented SDES following the standard structure: a 10-bit raw key is first permuted using P10, split into two halves, and left shifted to derive two 8 bit subkeys, K1 and K2, via P8. Encryption applies and initial permutation (IP) to the 8-bit plaintext, followed by two rounds of the function f_k (using K1 and K2, with a swap of the two halves between rounds), and finally the inverse initial permutation IP^{-1} . Decryption follows the same structure but applies the subkey in reverse order (K2 first, then K1).

The function f_k expands the right half of the 8 bits using the EP-table, XORs ut wuth the subkey, splits the result into two 4-bit halves, and looks each half up in the S0 and S1 substitution boxes (using the outer two bits as the row and the inner two bits as the column) before permuting the combined result with P4 and XORing the left half.

Running the implementation on the given test cases produced:

Encryption

0000000000	00000000	11110000
1111111111	11111111	00001111
0000011111	00000000	01000011
0000011111	11111111	11100001

Decryption:

1000101110	00011100	00111000
1000101110	11000010	00001100
0010011111	10011101	11111100
0010011111	10010000	10100101

2.2.2 Triple DES

I implemented Triple DES as a class wrapping the base DES function with two independent 10-bit keys k_1 and k_2 :

$$\text{Enc}_{3\text{DES}}(p) = \text{Enc}_{\text{DES}}(k_1, \text{Dec}_{\text{DES}}(k_2, \text{Enc}_{\text{DES}}(k_1, p)))$$

Decryption reverses this process:

$$\text{Dec}_{3\text{DES}}(c) = \text{Dec}_{\text{DES}}(k_1, \text{Enc}_{\text{DES}}(k_2, \text{Dec}_{\text{DES}}(k_1, c)))$$

Testing with the given test cases produced:

Triple DES

Encryption:

0000000000	0000000000	00000000	11110000
1000101110	0110101110	11010111	10111001
1000101110	0110101110	10101010	11100100
1111111111	1111111111	10101010	00000100

Decryption:

1000101110	0110101110	11111101	11100110
1011101111	0110101110	01001111	01010000
0000000000	0000000000	01010010	10000000

2.2.3 Cracking DES (ctx1)

Since DES uses only a 10-bit key, there are only $2^{10} = 1024$ possible keys, making brute-force search entirely feasible. I decrypted the ciphertext with every possible key and converted the resulting bit string into ASCII text 8 bits at a time. I created a simple check function that checks whether at least 95% of the decoded characters fall within the ASCII range (32-126). Only the correct key produced text that passed this readability check: This approach found a key "1111101010", decrypting to the message:simplifieddesisnotsecureenoughtoprovideyousufficientsecurity.

2.2.4 Cracking Triple DES (ctx2)

Since Triple DES uses two 10-bit keys, a brute-force search over all keys would require $2^{10} \times 2^{10} = 2^{20}$ decryption attempts per byte, which is not an effective approach. Instead I used a meet-in-the-middle approach. I computed two lookup tables, one mapping each of the 1024 possible keys, and all 256 possible

byte values to their SDES decryption and another one for encryption. For each k_1 , I decrypt every ciphertext byte once. Then for each candidate k_2 , I look up the middle encryption step and the final decryption step from the computed tables rather than recomputing them, reducing the table lookups per byte instead of 2^{20} full SDES computations. This approach found $k_1 = "1111101010"$ and $k_2 = "0101011111"$ with the message `simplifieddesisnotsecureenoughtoprovidey-
ousufficientsecurity`

Chapter 3

Implement Secure Communications

I implemented a simplified secure communication between Alice and Bob, following the three-part structure of key exchange, random key generation, and symmetric encryption.

3.0.1 Diffie-Hellman Key Exchange

Alice and Bob first agree on public parameters: a prime $p = 1187$ and a generator $g = 2$, where $p - 1 = 2 \times 593$ and 593 is itself prime, the safe-prime required for a cyclic group. Alice and Bob generate a private key a and b and compute their public keys:

$$x = g^a \bmod p, \quad y = g^b \bmod p$$

These public keys are exchanged publicly, allowing each party to compute the same shared secret independently:

$$K_{ab} = x^b \bmod p$$

3.0.2 Deriving a Symmetric Key with BBS

A Diffie-Hellman secret is not suitable as a symmetric key, therefore I used the Blum Blum Shub (BBS) generator as a strong pseudo-random number generator (CSPRNG) to derive the final key. BBS requires a modulus $n = P \cdot Q$ where P and Q are large primes satisfying $P \equiv Q \equiv 3 \pmod{4}$ (Blum primes). I generated two such primes in the range 400-1000, and used the shared secret K_{ab} as the seed. The generator squares the modulo n and extracts the least significant bit at each step producing a stream of pseudorandom bits. I generated 1+ bits to match the key length required by SDES.

3.0.3 Symmetric Encryption with SDES

With the derived key K , Alice encrypts her message using SDES, Bob having derived the same key K through his own key exchange and BBS computation, decrypts the ciphertext using the same SDES key and recovers Alice's original message demonstrating: $\text{Dec}(\text{Enc}(IN_A, K), K) = IN_A$.

3.0.4 Results

Running the program produced the following:

```
Alice, enter message to encrypt: Hello Bob, how are you?  
Alice sends ciphertext: 101000101100001101001111010011110011010010010  
00001000111001101001110111111110111001000000001110001101000011110010  
0100001000011001001100110000111001000000000100001101001001111101100110  
Bob decrypts ciphertext to: Hello Bob, how are you?
```

```
Public parameters: p = 1187, g = 2  
Alice's public key (PUa): 79  
Bob's public key (PUb): 1075  
Shared key Kab: 243  
Derived symmetric key K: 1000111100
```

Bob successfully recovered Alice's message after decrypting the ciphertext with the derived shared key, confirming that the key exchange, CSPRNG and symmetric encryption/decryption function correctly together.

Chapter 4

Conclusion

The objective was to implement and analyse simplified cryptographic primitives and use them. All tests for SDES and Triple SDES matched, and both ciphertexts in Task 2 were cracked, confirming the implementation of the assignment.